Technical Report — FastSpeech 2 (English and Telugu)

Status: draft. Sections 1–6 and section 11 are written from measured values that are recorded in `CHANGES.md`. Section 7 holds an interim reading of the English run and its final version, and sections 8 (listening), 9 and 10, need artefacts that do not exist yet — finished training, generated audio and variance plots. They are marked **PENDING**; no number or observation has been invented for them.

Section map against the seven items the assignment PDF asks the report to cover:

PDF item	Section here
Implementation approach, architecture components, modifications per language	1, 2, 3
Dataset selection, subsampling, preprocessing, split, training configuration	4, 5, 6
Loss-curve analysis: convergence, instability, overfitting	7 (interim; final **PENDING**)
Generated-speech observations per test sentence (pronunciation, quality)	8 (text side measured; listening **PENDING**)
Variance-adaptor analysis (duration, F0, energy) per test sentence	9 (**PENDING**)
English vs Telugu comparison, limitations, failures, improvements	10 (**PENDING**)
Reproducibility: dependencies, configs, commands	11

---

1. Approach

FastSpeech 2 is a feed-forward, non-autoregressive text-to-speech model. Instead of an attention-based autoregressive decoder, it predicts a mel-spectrogram in a single pass: the text is encoded by a Transformer encoder, a length regulator expands each text position to the number of frames it will occupy, and the decoder produces the mel-spectrogram directly. Because there is no attention to supply alignment, the model predicts the alignment itself with a **duration predictor**, and it also predicts two other "variance" signals that a mel-spectrogram alone would otherwise leave to the vocoder: **pitch (F0)** and **energy**. Each predicted value is embedded and added back to the expanded text representation by the variance adaptor before decoding. The original paper supplies the durational supervision by distillation from an autoregressive teacher; this assignment instead derives durations from forced alignment, which is the simpler and fully reproducible route.

Two independent models are trained from scratch, one per language. No pretrained acoustic-model weights are used. The only external pretrained component is the HiFi-GAN vocoder, which converts mel-spectrograms to waveforms; the assignment allows a pretrained vocoder and the reference repository ships one.

The starting point is `https://github.com/ming024/FastSpeech2` at commit `d4e79eb` (MIT licence). `LICENSE` and `THIRD_PARTY_NOTICES.md` are kept in the repository. Each language folder is self-contained: it carries its own copy of `audio/`, `model/`, `preprocessor/`, `text/`, `transformer/`, `utils/`, `dataset.py`, `evaluate.py`, `hifigan/` and its own `config.yaml`.

2. Architecture components taken from the reference implementation

Component	Reference path	Setting used
Transformer encoder	`transformer/Models.py`	4 layers, 2 heads, hidden 256, conv filter 1024, kernels `[9, 1]`, dropout 0.2
Transformer decoder	`transformer/Models.py`	6 layers, 2 heads, hidden 256, dropout 0.2
Variance adaptor and predictors	`model/variance_adaptor.py`, `model/modules.py`	filter 256, kernel 3, dropout 0.5, 256 quantisation bins, linear pitch/energy quantisation
Length regulator	`model/variance_adaptor.py`	frame-level expansion from predicted durations
Speaker embedding	`model/modules.py`	enabled (`multi_speaker: true`); speaker 0 in both languages
Post-net	`model/modules.py`	reference default
Acoustic model size	—	35,159,617 parameters (measured by the CPU smoke test)
Vocoder	`hifigan/`	HiFi-GAN **universal** generator, pretrained, 22.05 kHz, 80 mel bins, hop 256, `mel_fmax` 8000 Hz

Mel front-end: 22,050 Hz, `filter_length` 1024, `hop_length` 256, `win_length` 1024, 80 mel channels, F0 range `[0, 8000]` Hz. Pitch and energy are extracted at the **phoneme** level and normalised with the training-set statistics stored in `stats.json`. `max_seq_len` is 1000 frames, with a margin of 20 frames imposed by the preprocessor, so a clip whose mel exceeds 980 frames (250,879 samples ≈ 11.38 s) is dropped rather than silently truncated at training time.

3. Modifications to the reference implementation

Both languages:

- **One configuration file per language** with the four sections the assignment requires (`preprocess`, `model`, `train`, `inference`), replacing the reference's three YAML files. `utils/config.py` loads and validates it, so a bad value fails before a long job starts.
- **Manifest-driven preprocessing** instead of the reference's random split: `manifests/train.csv` and `manifests/val.csv` are fixed inputs, built once with a fixed seed, and no clip appears in both.
- **Full drop accounting**: every clip that cannot be used is classified (alignment, missing TextGrid, missing audio, pitch, feature statistics, transcript, too long) and written to `preprocessing_log.csv`.
- **Training instrumentation**: fixed seed, per-step CSV loss logs (`logs/train/train_loss.csv`, `logs/val/val_loss.csv`), configurable `log_step` / `val_step` / `synth_step` / `save_step`, checkpoint resume that restores RNG state, and a final checkpoint written even when `total_step` is not a multiple of `save_step`.

- **`inference.py`** replaces the reference's interactive `synthesize.py` single-sentence mode: it reads `docs/test_sentences.json`, refuses to run while any sentence's `final` field is null, and writes `<ID>.wav` to `Results/<Language>/test_generated_audios/`.

English:

- Reference English frontend kept (ARPAbet lexicon + G2P), with a rule-based normaliser for numbers, dates, times and abbreviations, and an optional supplementary G2P dictionary for proper names. The normalisation is applied to the transcript only; the stored test sentences are not edited.
- Forced alignment with MFA `english_us_arpa`. Out-of-vocabulary rate on the corpus was 4.47 %, handled with the bundled G2P model.

Telugu:

- **Character-level frontend** (`text/telugu.py` + a frozen Telugu symbol table in `text/symbols.py`): NFC normalisation, whitespace and format-control handling, punctuation folding, and explicit unknown-character errors. The symbol IDs come from an explicit code-point range list rather than `unicodedata`, so they do not change with the Python version.
- **No Telugu lexicon or phoneme set was invented.** A grapheme-dictionary MFA aligner was trained on the corpus itself (each word mapped to its space-separated characters) to obtain phone-level durations on the character tier.
- The preprocessor requires `len(text_to_sequence(text)) == len(duration)`; the `.lab` used for each clip therefore contains the alignment text, not the raw dataset transcript.
- Telugu number/date/time normalisation in `T4` is **not implemented**; see §9 for how this is reported as a failure rather than hidden. *(PENDING confirmation against the final frontend behaviour.)*

4. Datasets, selection and subsampling

English — Svarah (`ai4bharat/Svarah`)

AI4Bharat's Indian-accented English read/spontaneous benchmark, built for ASR rather than TTS. Two speakers were exported (one female, one male): 3,276 + 2,827 = 6,103 clip/transcript pairs. The manifests that entered preprocessing hold 5,582 clips, 6.078 h of audio before alignment (train 5.132 h, val 0.946 h). Two speakers is a deliberate compromise: Svarah averages only a few minutes per speaker across ~117 speakers, so a single-speaker model is impossible and the reference speaker embedding is used instead.

Stage	Clips	Audio
Exported from the Hub	6,103	—
Entered the manifests	5,582	6.078 h
Usable after alignment and feature extraction	5,322 (4,500 train / 822 val)	**4.842 h**

Drops: 260 in total — alignment 164 (non-positive phone duration), missing TextGrid 71, pitch extraction 20 (no voiced frames), feature statistics 4, transcript 1. The gap between 6.078 h of

manifest audio and 4.842 h of usable audio is audio outside the phone tier plus the dropped clips; 4.842 h is the honest training figure.

Telugu — IndicTTS Telugu (`SPRINGLab/IndicTTS_Telugu`)

A Hugging Face copy of the IIT Madras IndicTTS data: 8,576 clips, 36.71 h, two speakers (male 12.79 h / 4,533 clips; female 23.92 h / 4,043 clips). The **male speaker only** was used, so the model sees one voice and a single speaker embedding; a single speaker gives the cleanest comparison against the two-speaker English model, and the male subset is the smaller and easier-to-balance one. Subsampling keeps clips inside the duration window implied by the 11.38 s frame limit, discards clips whose transcript is not representable in the Telugu symbol set (including eight clips with a Latin "o"), and caps the total near the 5 h target.

Stage	Clips	Audio
Source rows (male)	4,533	12.79 h
Entered the manifests	3,363	—
Usable after alignment and feature extraction	3,269 (2,751 train / 518 val)	**4.700 h**

Drops: 94 — alignment 83, missing TextGrid 3, missing audio 8.

> **Known limitation, stated explicitly.** The Telugu audio in this run is **16 kHz upsampled to 22,050 Hz** (`resample_poly`, 441/320, PCM 16-bit). The 22.05 kHz export was lost when the Kaggle session working directory was reset, and the surviving aligner corpus is 16 kHz. Because `mel_fmax` is 8000 Hz, no mel band above 8 kHz is used, so the upsampling does not add information but also does not remove any band the model actually sees. The vocoder still receives a 22.05 kHz sampling rate and therefore produces energy above 8 kHz that the acoustic model did not predict.

5. Preprocessing and alignment

Step	English	Telugu
Aligner	MFA 3.4.3, `english_us_arpa` (acoustic + dictionary + G2P for OOV)	MFA 3.4.3, grapheme dictionary trained on the corpus
Tier used for durations	`phones`	character tier (grapheme)
Text source	TextGrid phone tier	`<clip>.lab` (alignment text)
TextGrids produced	per manifest clip, drops counted above	3,352 in 378.5 s
Audio	resample to 22,050 Hz, mono, `max_wav_value` 32768	same (from 16 kHz source, see limitation)
Features	mel 80 × T, phoneme-level pitch and energy, durations from the alignment	same
Normalisation	training-set mean/σ from `stats.json`	same
Output layout	flat: `mel/<speaker>-mel-<clip>.npy`, same for `pitch`, `energy`, `duration`	same

English `stats.json`: pitch `mean 177.64, std 56.06, min -2.0096, max 9.0707`; energy `mean 27.01, std 26.67, min -1.0123, max 8.1121` (both in normalised units).

Splits: fixed seed, clip-level and transcript-level disjointness verified, every validation speaker also present in training. Manifests are committed under `English/manifests/` and `Telugu/manifests/`.

6. Training configuration

Setting	English	Telugu
Steps	60,000	**PENDING** (measured on the Telugu calibration run)
Batch size × grad-accum	16 × 1	16 × 1
Steps per epoch	281 (4,500 clips)	172 (2,751 clips)
Optimiser	Adam, β 0.9/0.98, ε 1e-9, weight decay 0	same
Learning rate	Noam schedule on `256^-0.5`: peak ≈ 9.9e-4 at step 4,000, ×0.3 at 20k/26k/33k, ≈ 6.9e-6 at 60k	same schedule, anneal points pending
Warm-up	4,000 steps	4,000 steps
Gradient clipping	1.0	1.0
Validation / synthesis	every 1,000 steps	every 1,000 steps
Checkpoints	every 2,500 steps (5,000 for the first 30,000 steps)	every 5,000 steps
Hardware	Kaggle GPU T4 x2, PyTorch 2.11.0+cu130, `nn.DataParallel` (batch 16 = 8+8)	same once the GPU slot frees
Measured step time	**0.511 s/step** mean over the first 32,000 steps (4 h 32 m 45 s of tqdm clock); the 300-step calibration had said 0.304 s/step	**PENDING**
Wall clock	≈ 8.5 h for 60,000 steps at the measured mean, not the 5.3 h the calibration implied	**PENDING**
Budget rationale	60,000 steps ≈ 213 epochs; the reference 900,000 steps is unattainable and would leave the learning rate flat, so the anneal points were scaled to keep the reference fractions (33/43/55 %)	same logic once the step time is measured

Run interruption at step 32,000. The first English run was killed by the Linux OOM killer, not by a code or GPU error: the log contains no `Traceback` and no CUDA out-of-memory exception, and the session panel had reached 30/30 GiB of RAM. The step rate also degraded as it went — the last 2,000 steps ran at ~0.65 s/step against ~0.30 s/step at the start — which is the same memory growth showing up as time. Checkpoints up to `30000.pth.tar` were on disk and survived, so the run resumes from 30,000 rather than restarting. Two changes were made before resuming, neither of them to the model, the data or the schedule: `train.step.save_step` was cut to 2,500 so a future kill costs ~21 minutes instead of ~45, and `utils/tools.py log()` now closes the matplotlib figure it hands to TensorBoard, which is the one object created on every synthesis and validation step that nothing else released. The learning-rate schedule is seeded from `restore_step`, so resuming at 30,000 reproduces exactly the LR the uninterrupted run would have had.

7. Loss analysis — interim reading (steps 100–25,000); final version PENDING

Source: `logs/train/log.txt` and `logs/val/log.txt` copied out of the running Kaggle session when training had reached step 25,000 of 60,000. The final version of this section will use the complete logs and the two PNG plots.

7.1 Training curve

Total loss falls from 10.0987 at step 100 to 1.1886 at step 25,000. The last twenty logged points average 1.3096 with a standard deviation of 0.0810, so the curve is smooth. The only large jump is 2.21 between steps 200 and 300, at the end of warm-up; there is no divergence and no loss explosion anywhere in the range.

Step	Total	Mel	Post-net	Pitch	Energy	Duration
2,000	3.4646	1.0418	1.0928	0.5762	0.5365	0.2174
6,000	2.3361	0.7680	0.7685	0.3319	0.2925	0.1752
10,000	2.5023	0.8076	0.8076	0.3041	0.3903	0.1927
14,000	1.6698	0.6822	0.6819	0.1023	0.1050	0.0984
18,000	1.5338	0.5451	0.5450	0.1586	0.1272	0.1578
22,000	1.3200	0.5782	0.5778	0.0750	0.0464	0.0426
25,000	1.1886	0.4841	0.4838	0.0626	0.0475	0.1105

7.2 Validation curve

Validation loss reaches its minimum early and then rises:

Step	Total	Mel	Pitch	Energy	Duration
1,000	3.8787	1.0210	0.6174	0.9921	0.2248
5,000	3.8918	0.9946	0.6335	1.0459	0.2262
7,000	**3.8246**	0.9835	0.6343	0.9999	0.2235
15,000	4.0304	1.0044	0.6939	1.0697	0.2583
20,000	4.0585	1.0127	0.6882	1.0846	0.2604
25,000	4.0588	1.0142	0.6811	1.0780	0.2715

7.3 Convergence, instability, overfitting (interim)

- **Convergence.** Training converged normally; by step 25,000 the total loss is 8.5× lower than at step 100 and the post-net loss has almost caught up with the mel loss, which is the expected behaviour once the post-net stops adding correction on top of a well-predicted mel.
- **Instability.** None after warm-up. The pitch loss oscillates between about 0.05 and 0.24 across neighbouring thousand-step blocks (e.g. 0.0497 at 24,000 and 0.0626 at 25,000, against 0.2376 at 24,900), which is ordinary mini-batch variance, not divergence.

- **Overfitting.** Validation loss is now 6.1 % above its best value and has risen monotonically since step 7,000, while training loss fell from 3.44 to 1.19 over the same span. By step 25,000 the training set has been traversed about 89 times (281 steps/epoch, 4,500 training clips). The rise is concentrated in the variance predictors — duration +21 %, energy +9 %, pitch +10 % from step 1,000 to 25,000 — while the mel losses stay flat near 1.01. In other words the acoustic model stops generalising after roughly 25 epochs and the predictors begin to memorise this speaker pair's timing and dynamics.
- **Consequence for checkpoint selection.** Validation-loss minimum is step 7,000, but checkpoints are written every 5,000 steps, so the best available candidates are 5,000, 10,000 and the final one. The final choice will be made by comparing those three on the test sentences, not by validation loss alone; the reason will be stated in the final version of this section.

Log quality note: eight training log lines (steps 12,100–12,800) are absent from the copied file — a buffering artefact of reading the file while training wrote to it. Training itself is continuous, since the validation entries for steps 12,000 and 13,000 are present.

8. Test-sentence observations — text side measured, listening side PENDING

The sentences come from `docs/test_sentences.json`, field `final`, unmodified. Five per language: E1, E2a, E2b, E3, E4 and T1, T2a, T2b, T3, T4. The listening-based part of this section (pronunciation, overall quality) is pending; the text-processing part is measured and reproduced below.

8.0 The ten strings the models receive

Stored in `docs/test_sentences.json` under `final` and used unchanged by `inference.py`. Each line below was written by a script that read the JSON, not re-typed from the PDF, so the Telugu column is byte-identical to the text the model receives.

ID	Sentence
E1	The antidisestablishmentarianism movement gained considerable momentum during the parliamentary proceedings.
E2a	The orchestra's conduct during the ceremony was praised by the audience.
E2b	The researchers will conduct an experiment to evaluate the proposed method.
E3	The rhythm of the orchestra changed unexpectedly.
E4	The meeting will start at 10:45 AM on 12 November 2026, so andaroo should arrive on time.
T1	న్యూరోఫార్మకోలాజికల్ పరిశోధనలో నూతన ఔషధాల ప్రభావాన్ని విశ్లేషించారు.
T2a	తాడుపై అతని పట్టు చాలా బలంగా ఉంది.
T2b	ఆమె పట్టు చీరను ధరించి వేడుకకు వచ్చింది.
T3	పులి అడవిలో నిశ్శబ్దంగా నడిచింది.
T4	ఈ సమావేశం 2026 నవంబర్ 12వ తేదీ ఉదయం 10:45 గంటలకు ప్రారంభమవుతుంది, సమయానికి సిద్ధంగా ఉంచండి.

> **Font requirement (measured).** The Telugu rows need a Telugu-capable font. This PDF was built on the author's Windows machine, where `Nirmala.ttc` supplies the glyphs: the rendered page shows all five Telugu sentences correctly and the check in `scripts/render_report_pdf.py` counted 223 of the 220 Telugu codepoints in the Markdown inside the PDF, with no replacement characters. The same build on a Linux box with no Indic font — including the Kaggle image — renders those rows as blanks, so either build the report locally or pass `--telugu-font <path>` to embed one.

8.1 What the frontends actually produce (measured)

ID	Symbols	Text the model receives after normalisation
E1	108	`the antidisestablishmentarianism movement gained considerable momentum during the parliamentary proceedings.`
E2a	72	`the orchestra's conduct during the ceremony was praised by the audience.`
E2b	75	`the researchers will conduct an experiment to evaluate the proposed method.`
E3	49	`the rhythm of the orchestra changed unexpectedly.`
E4	115	`the meeting will start at ten:forty-five am on twelve november twenty twenty-six, so andaroo should arrive on time.`
T1	68	unchanged Telugu characters (character-level frontend, no lexicon)
T2a	34	unchanged
T2b	40	unchanged
T3	33	unchanged
T4	91	**unchanged, including the digits**: `… 2026 … 12ఏ … 10:45 …`

All ten encode without an unknown-character error, which is the precondition for inference.

Two consequences worth stating before listening:

- **E4 is normalised correctly.** `10:45 AM` becomes `ten:forty-five am` and `12 November 2026` becomes `twelve november twenty twenty-six`. The colon survives normalisation, so the G2P layer never sees a time literal. `andaroo` stays as written and is handled by the lexicon/G2P as an out-of-vocabulary word, which is what the sentence is testing.
- **T4 is not normalised.** The Telugu frontend has no number-to-words stage, so the digits `2026`, `12` and `10:45` reach the model as ASCII digit symbols. Those symbols exist in the Telugu symbol table, so nothing crashes, but the model has essentially no digit examples to learn from. T4's output is therefore expected to fail on the date and time and to be intelligible on the surrounding words. This is a frontend gap, not a training failure, and it is reported as such rather than hidden.

Why each sentence is interesting:

ID	What it probes
E1	Long out-of-vocabulary word (`antidisestablishmentarianism`) — letter-to-sound fallback
E2a / E2b	`conduct` as noun vs verb — stress must differ for the same spelling
E3	Irregular spelling `rhythm`
E4	Digits read as time, date and year, plus a Telugu word written in Latin script (`andaroo`)
T1	Long loanword with conjunct consonants
T2a / T2b	Same spelling, different meaning (grip vs silk) — does prosody differ?
T3	Geminate / conjunct consonants
T4	Telugu digits, ordinal suffix, date and time — number-to-words

9. Variance-adaptor analysis — PENDING

For each sentence and each of duration, pitch and energy the report will state the predicted total duration in frames and seconds, the mean and range of predicted F0, and the range of predicted energy, read from the saved predictions, and then explain in words why the pattern is plausible or not (e.g. phrase-final lengthening, the effect of a rare long word on per-symbol durations, flat F0 where a question-like contour would be expected).

10. English vs Telugu comparison — PENDING

To be written after both models are trained and both sets of samples exist. Planned structure: data-side differences (two speakers vs one, read vs read/spontaneous, 16 kHz upsampled Telugu, English lexicon + G2P vs Telugu grapheme model), what that predicts for pronunciation accuracy (English OOV names and loanwords vs Telugu conjuncts and digits), and which failures are attributable to data, to the frontend, or to the training budget.

11. Reproducibility

Dependencies are listed in `requirements.txt`, which is a **proposed pin set, not a record of the environment that ran**: the file was never installed as a whole on Kaggle. The sessions that produced these results used the Kaggle image's own scientific stack on Python 3.13 with PyTorch 2.11.0+cu130, plus targeted installs of `pyworld`, `tgt`, `unidecode` and `inflect`; `g2p_en` was never installed, so the supplementary name dictionary was not used. Each language folder has one `config.yaml`; every path inside it is relative to that folder, so all commands are run from inside `English/` or `Telugu/`.

```
# dependencies
pip install -r requirements.txt

# 0. vocoder weights (shipped zipped to stay under GitHub limits)
cd English && unzip -o hifigan/generator_universal.pth.tar.zip -d hifigan

# 1. preprocessing (needs the aligned corpus: TextGrids for English, .lab + TextGrids for Te
python preprocess.py config.yaml --allow-dropped

# 2. training (checkpoints -> checkpoints/, CSV loss logs -> logs/)
python train.py -c config.yaml
python train.py -c config.yaml --restore_step 30000 --resume

# 3. plots
python scripts/plot_loss.py --config config.yaml
python scripts/plot_variance.py --config config.yaml --restore_step <STEP>

# 4. inference on the five test sentences
python inference.py --config config.yaml --restore_step <STEP>
```

Environment used for training: Kaggle notebook on **GPU T4 x2**, Python 3.13, PyTorch 2.11.0+cu130. `train.py` wraps the model in `nn.DataParallel` unconditionally, so the batch of 16 is split 8/8 across both cards; both GPUs report 100 % utilisation for a single training process. Forced alignment ran in a separate MFA 3.4.3 environment installed from conda-forge, invoked as `PATH=<env>/bin:$PATH <env>/bin/mfa ...` because a session restore had stripped the environment's exec bits and shared-library symlinks.

Appendix A — measured figures used above

Quantity	English	Telugu
Clips in manifests	5,582	3,363
Usable clips	5,322 (4,500 / 822)	3,269 (2,751 / 518)
Usable hours	4.842	4.700
Speakers	2	1
Dropped	260	94
Params	35,159,617	35,159,617
Vocoder	HiFi-GAN universal (pretrained)	same